Master Theorem for Divide and Conquer Recurrences

According to the master theorem, the runtime of the above algorithm can be expressed as:

$$\mathbf{T(n) = aT(n/b) + f(n)}$$

where n = size of the problem
a = number of subproblems in the recursion and a >= 1
n/b = size of each subproblem
f(n) = cost of work done outside the recursive calls like dividing into subproblems and cost of combining them to get the solution.

This theorem is an advanced version of the master theorem that can be used to determine the running time of divide and conquer algorithms if the recurrence is of the following form:-

$$\mathbf{T(n) = aT(n/b) + \theta(n^c log^p n)}$$

where n = size of the problem
a = number of subproblems in the recursion and a >= 1
n/b = size of each subproblem
b > 1, c >= 0 and p is a real number.

Then,

Case 1: if $a > b^c$, then $T(n) = \theta(n^{\log_b a})$

Case 2: if $a = b^c$, then
(a) if $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
(b) if $p = -1$, then $T(n) = \theta(n^{\log_b a} \log\log n)$
(c) if $p < -1$, then $T(n) = \theta(n^{\log_b a})$

Case 3: if $a < b^c$, then
(a) if $p >= 0$, then $T(n) = \theta(n^c \log^p n)$
(b) if $p < 0$, then $T(n) = \theta(n^c)$

Examples

Example 1:
$T(n) = T(n/2) + O(1)$
$a = 1, b = 2, c = 0$ and $p = 0$
$b^c = 1$. So, $a = b^c$ and $p > -1$ [Case 2.(a)]
$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
$T(n) = \theta(\log n)$

Example 2:
$T(n) = 2T(n/2) + O(n)$
$a = 2, b = 2, c = 1, p = 0$
$b^c = 2$. So, $a = b^c$ and $p > -1$ [Case 2.(a)]
$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
$T(n) = \theta(n \log n)$

Example 3:
$T(n) = 3T(n/2) + n^2$
$a = 3, b = 2, c = 2, p = 0$
$b^c = 4$. So, $a < b^c$ and $p = 0$ [Case 3.(a)]
$T(n) = \theta(n^c \log^p n)$
$T(n) = \theta(n^2)$

Example 4:
$T(n) = 3T(n/2) + \log^2 n$
$a = 3, b = 2, c = 0, p = 2$
$b^c = 1$. So, $a > b^c$ [Case 1]
$T(n) = \theta(n^{\log_b a})$
$T(n) = \theta(n^{\log_2 3})$

Example 5:
$T(n) = 2T(n/2) + n\log^2 n$
$a = 2, b = 2, c = 1, p = 2$
$b^c = 2$. So, $a = b^c$ [Case 2.(a)]
$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
$T(n) = \theta(n^{\log_2 2} \log^3 n)$
$T(n) = \theta(n \log^3 n)$

Example 6:
$T(n) = 2^n T(n/2) + n^n$
This recurrence can't be solved using above method since function is not of form $T(n) = aT(n/b) + \theta(n^c \log^p n)$

Master Theorem for Subtract and Conquer Recurrences

Let T(n) be a function defined on positive n as shown below:

$$T(n) = aT(n-b) + f(n)$$

for some constants c, a>0, b>0, c>=0 and function f(n). If f(n) is $O(n^c)$, then
1. If a<1 then $T(n) = O(n^c)$
2. If a=1 then $T(n) = O(n^{c+1})$
3. if a>1 then $T(n) = O(n^c a^{n/b})$

Examples

Example 1:
T(n) = 3T(n-1) + 1
a=3, b=1, f(n)=1 so c=0;
Since a>1, $T(n) = O(n^c a^{n/b})$
$T(n)= O(n^0 3^{n/1})$
$T(n)= 3^n$

Example 2:
T(n) = T(n-1) + n(n-1)
a=1, b=1, f(n)=n(n-1) so c=2;
Since a=1, $T(n) = O(n^{c+1})$
$T(n)= O(n^{2+1})$
$T(n)= O(n^3)$

Example 3:
T(n) = 2T(n-1) – 1
This recurrence can't be solved using the above method
since function is not of form T(n) = aT(n-b) + f(n)

Example 3:
T(n) = 0.5T(n-1) + n
a=0.5, b=1, f(n)=n so c=1;
Since a<1, $T(n) = O(n^c)$
$T(n)= O(n^1)$
T(n)= O(n)